

Technical Specification: Bilingual AI Search Infrastructure (ETL Pipeline + Search API)

1. Executive Summary

We are building a bilingual (Hungarian & English) search infrastructure for a B2B marketplace. The system consists of two independent Python services:

1. **ETL Pipeline:** Ingests ~38,000 categories and ~10,000 company profiles from internal PostgreSQL, generates language-specific **Narrative Templates**, performs AI enrichment, and stores hybrid indexes (Vector + BM25) in `pgvector`.

2. **Search Microservices:** A FastAPI service providing low-latency hybrid search using **Reciprocal Rank Fusion (RRF)** to blend dense vector similarity (OpenAI) and lexical matching (BM25).

Core Constraint: The system must treat Hungarian and English as first-class citizens with equal semantic depth, ensuring no context is lost during vectorization while maintaining strict keyword precision for technical specs.

2. Architecture & Tech Stack

- **Language:** Python 3.10+
 - **Database:** PostgreSQL 15+ with `pgvector` extension.
 - **Embeddings:** OpenAI `text-embedding-3-small` (1536 dimensions).
 - **LLM (Enrichment):** OpenAI `gpt-4o-mini`.
 - **NLP/Lemmatization:**
 - Hungarian: `huspacy` (`hu_core_news_lg`).
 - English: `spaCy` (`en_core_web_lg`).
 - **API Framework:** FastAPI (preferred) or Django REST Framework.
 - **Infrastructure:** Dockerized deployable units (ETL container + API container).
-

3. Module 1: The ETL Pipeline

The ETL is the "brain" of the system. It must be idempotent, hash-based (cost-saving), and strictly bilingual.

3.1. General Logic (The "Dual Path" Rule)

For every record (Category or Company), the pipeline must generate two parallel representations:

1. **Embedding Path (Semantic):** Uses the **RAW Narrative Template** (natural sentences).
 - *Constraint:* ZERO lemmatization here. Context like "kínálunk" (we offer) vs. "kínál" (offers) must be preserved for the transformer model.
2. **BM25 Path (Lexical):** Uses **Lemmatized Tokens**.
 - *Constraint:* Uses `huspacy` (HU) / `spaCy` (EN).
 - *Constraint:* **Protected Terms Pass** is mandatory before lemmatization (see 3.4 below).

3.2. Pipeline A: Category Processing (38k records)

- **Input:** Source DB (name, synonyms, hierarchy).
- **Process:**
 1. **Hash Check:** Skip if `md5(content)` hasn't changed.

2. **Translation:** If source is HU, translate to EN (and vice versa) preserving technical terms.

3. **AI Enrichment (gpt-4o-mini):** Generate 6 mandatory fields per language:

- `ai_type` (7-level classification: e.g., `GÉP_ÉS_BERENDEZÉS`).
- `status` (OK/REVIEW/DELETE).
- `add_words` (SEO keywords).
- `recommended_headline` (Marketing title).
- `synthetic_questions` (3 user-centric questions, e.g., "What load capacity...").
- `topic_suggestions` (Data quality flags).

4. **Narrative Template Construction:**

- *Format:* "Electric forklift category, which belongs to the machinery group.
Also known as: pallet truck..." (Natural language, not labeled JSON).

5. **Indexing:** Insert into `category_vectors` table (one row per language per category).

3.3. Pipeline B: Company Processing (5k-10k records)

• **Input:** Company profile, products (with properties), blog posts.

• **Process:**

1. **HTML Cleaning:** Remove scripts, styles, navbars (using BeautifulSoup, NOT LLM).
2. **Narrative Assembly:** Combine all data into a coherent story.
3. **Synthetic Questions:** Generate 3-5 buyer questions (e.g., "Do you do custom CNC machining?") to bridge the problem-solution gap.

4. **Smart Chunking:**

- Max 500 words per chunk.

• **Context Header Injection (CRITICAL):** Every chunk *must* start with `[COMPANY : Name] [PROFILE: Main Category]` so the chunk is standalone-searchable.

5. **Atomic Swap:** Use a transaction to `DELETE` old chunks and `INSERT` new ones to prevent data corruption during updates.

3.4. Mandatory Feature: "Protected Terms Pass"

To prevent the lemmatizer from destroying brand names (e.g., "Bunting" becoming "bunt") or technical codes (e.g., "ISO-9001"), you must implement a pre-processing filter:

- **Logic:** Before passing text to `huspacy/spaCy`, check against a `brands_whitelist.json` and Regex patterns (for alphanumerics like "CNC-100").
- **Action:** Temporarily mask these terms or force the lemmatizer to ignore them.

4. Module 2: Search Microservices (API)

The API exposes endpoints for the frontend (Dify chatbot) and backend (RFQ matching).

4.1. Core Search Logic (Hybrid + RRF)

Do not use weighted averages (e.g., $0.7 * \text{vector} + 0.3 * \text{BM25}$). We require **Reciprocal Rank Fusion (RRF)**.

• **Algorithm:** $\text{score} = 1 / (k + \text{rank_vector}) + 1 / (k + \text{rank_bm25})$

where `k=60` (configurable).

• **Language Handling:**

- Detect query language (e.g., `langdetect`).
- Route to specific indexes: `bm25_tokens_hu` vs `bm25_tokens_en`.
- **Fallback:** If results < threshold, trigger search in the secondary language.

4.2. Endpoints

1. `POST /search/category` (**Low Latency**)

- *Goal:* Identify user intent (e.g., "Forklift").

- *SLA*: < 500ms (p95).
- *Logic*: Hybrid search on `category_vectors`. Returns `ai_type` and metadata.
- 2. **POST /match/companies (Deep Matching)**
 - *Goal*: Find best suppliers for an RFQ.
 - *Logic*:
 1. **Pre-filtering**: Apply JSONB filters (e.g., location, logistics capabilities).
 2. **Hybrid Search**: RRF on `company_vectors`.
 3. **Partner Boost**: Multiply score by 1.15 if `is_highlighted=true`.
 4. **Aggregation**: Return best chunks grouped by Company ID.
- 3. **POST /search/single-company (Whitelabel)**
 - *Goal*: Search *only* within a specific company's data (for partner website chatbots).
 - *Logic*: Strict `WHERE company_id = X` filter before vector search.

5. Acceptance Criteria & Testing

The project is considered complete only upon passing these specific tests:

1. **The "Bunting" Test (Protected Terms):**
 - Query: "Bunting gloves".
 - Pass: System finds records containing "Bunting" via BM25 exact match, proving the lemmatizer didn't corrupt the brand name.
2. **The "Atomic Swap" Test:**
 - Action: Kill the ETL process mid-update for a company.
 - Pass: The DB remains in the *old* state (no partial/corrupt data).
3. **The "Narrative" Test:**
 - Action: Inspect vector input logs.
 - Pass: Input must be natural sentences (e.g., "This company specializes in..."), NOT JSON or keyword lists.
4. **Bilingual Parity:**
 - Query: "Targonca" (HU) and "Forklift" (EN).
 - Pass: Both return semantically equivalent, high-quality results.

6. Deliverables

1. **Codebase**: Two deployable units (ETL package, Search API) in Python 3.10+.
2. **Docker**: `docker-compose.yml` for local dev (DB + ETL + API).
3. **Documentation**: `README.md` with setup, env vars, and API docs (Swagger/ReDoc).
4. **Test Suite**: End-to-end pytest script verifying the criteria above.

7. Next Steps for Proposal

Please update your scope and quote to reflect:

- **Timeline**: We are targeting a **7-week delivery**.
- **Confirmation**: Explicitly confirm understanding of the **Narrative Template, Protected Terms**, and **RRF** requirements.